

Reminder: Submission Instructions

1) First create a copy of this notebook in your drive and rename it so that it contains the course number (ISTA322), the semester (Sp22 for Spring 2022, Su22 for Summer 2022, and Fa22 for Fall 2022), the assignment code (HW1 for this assignment) and your name.

e.g. my copy for Spring 2023 would be ISTA_322_Sp23_HW3_dan_charbonneau)

2) When you are ready to submit. Prepare three files: the python file (File->Download->Download .py), the notebook file (File->Download->Download .ipynb), and **PDF** version of your notebook (**after running all cells**). You should be able to create a pdf through File -> print -> Destination: save as pdf

3) Create a new folder named firstname_lastname_hw1 (e.g my folder would be dan_charbonneau_hw3) put all three files in it. Compress (Zip) the folder you created with the files inside of it and submit this .zip file to D2L

incorrect filenames or submission formats will result in a loss of 50% of your grade

✓ HW3 - SQL

This homework has you working with a new database of information on ticket sales for various types of events. Your job will be to do some initial exploring and then demonstrate your ability to do all the different types of SQL queries we learned over the last two week. You'll also need to make one function that'll make looking at the tables easier.

These questions are written in the way someone would ask them to you. In other words, I'm using 'plain english' questions vs. ones where I'm very explicit in terms of what columns and tables to use. Your exploring of the database and functions to ease that process will come in handy here!

The database has been created using a set of data from Amazon. You can read more about what each table contains here: https://docs.aws.amazon.com/redshift/latest/dg/c_sampledb.html.

✓ Libraries and import functions

First bring the libraries we'll need!

```
import psycopg2
import pandas as pd
```

Now bring in all our functions we used in the lessons!

```
# Make our connection/cursor function
AWS_host_name = "ista322courserdb.chk8kg2yo4p6.us-east-1.rds.amazonaws.com"
AWS_dbname = "ista322courserdb"
AWS_user_name = "postgres"
AWS_password = "orangecatgoingnutsinacardboardbox"

def get_conn_cur(): # define function name and arguments (there aren't any)
    # Make a connection
    conn = psycopg2.connect(
        host=AWS_host_name,
        database=AWS_dbname,
        user=AWS_user_name,
        password=AWS_password,
        port='5432')

    cur = conn.cursor()    # Make a cursor after

    return(conn, cur)    # Return both the connection and the cursor

# Same run_query function
def run_query(query_string):

    conn, cur = get_conn_cur() # get connection and cursor

    cur.execute(query_string) # executing string as before

    my_data = cur.fetchall() # fetch query data as before

    # here we're extracting the 0th element for each item in cur.description
    colnames = [desc[0] for desc in cur.description]

    cur.close() # close
    conn.close() # close

    return(colnames, my_data) # return column names AND data

# Column name function for checking out what's in a table
def get_column_names(table_name): # arguement of table_name
    conn, cur = get_conn_cur() # get connection and cursor

    # Now select column names while inserting the table name into the WERE
```

```

column_name_query = """SELECT column_name FROM information_schema.columns
                        WHERE table_name = '%s' """ %table_name

cur.execute(column_name_query) # execute
my_data = cur.fetchall() # store

cur.close() # close
conn.close() # close

return(my_data) # return

# Check table_names
def get_table_names():
    conn, cur = get_conn_cur() # get connection and cursor

    # query to get table names
    table_name_query = """SELECT table_name FROM information_schema.tables
                        WHERE table_schema = 'public' """

    cur.execute(table_name_query) # execute
    my_data = cur.fetchall() # fetch results

    cur.close() #close cursor
    conn.close() # close connection

    return(my_data) # return your fetched results

```

✓ Q1 Make a SQL head function - 5 point

Make function to get the pandas equivalent of `.head()`

This function should be called `sql_head` and take a single argument of `table_name` where you specify the table name you want the head information from. It should return the column names along with the first five rows of the table along.

For full points, return a pandas dataframe with this information so it displays nicely :)

```

## Q1 Your function starts here
def sql_head(table_name, n=5):
    query = f"SELECT * FROM {table_name} LIMIT {n};"
    colnames, my_data = run_query(query)
    df = pd.DataFrame(my_data, columns=colnames)

    return df

# make sql_head function
get_table_names()

[('movie_sales',),

```

```
( 'rollup', ),
( 'creators', ),
( 'teams_df', ),
( 'cases', ),
( 'season_stats_df', ),
( 'spotify_features', ),
( 'death_data', ),
( 'judges', ),
( 'songs', ),
( 'referees_df', ),
( 'sales', ),
( 'venue', ),
( 'event', ),
( 'users', ),
( 'category', ),
( 'listing', ),
( 'date', ),
( 'game_data_df', ),
( 'movies', ),
( 'ratings', ),
( 'casetype', )]
```

Check that it works!

```
sql_head(table_name = 'sales')
```

Q1 Your function ends here - Any code outside of these start/end markers won't be graded

	sales_id	list_id	seller_id	buyer_id	event_id	date_id	qty_sold	price_paid	cc
0	2	4	8117	11498	4337	1983	2	76	
1	3	5	1616	17433	8647	1983	2	350	
2	4	5	1616	19715	8647	1986	1	175	
3	5	5	17433	11498	8647	1983	2	350	

✓ Q2 Explore and SELECT - 5 point

Let's start this homework with some basic queries to get a look at what's in the various tables. I want you to do the following.

- Use your `get_table_names()` function to see what tables are in the database.
- Use your `get_column_names()` to get the column names of each of the tables. **Do this all within a single cell to keep it neat.**
- Make and run a query that selects all columns from the event table. Only return the first 5 rows.
- Use the `sql_head()` function you created to get the first five rows of the sales table.

```
## Q2 Your function starts here
```

```
# Getting table names
```

```
get_table_names()
```

```
[('movie_sales',),  
 ('rollup',),  
 ('creators',),  
 ('teams_df',),  
 ('cases',),  
 ('season_stats_df',),  
 ('spotify_features',),  
 ('death_data',),  
 ('judges',),  
 ('songs',),  
 ('referees_df',),  
 ('sales',),  
 ('venue',),  
 ('event',),  
 ('users',),  
 ('category',),  
 ('listing',),  
 ('date',),  
 ('game_data_df',),  
 ('movies',),  
 ('ratings',),  
 ('casetype',)]
```

```
# Getting column names
```

```
movie_cols = get_column_names(table_name='movie_sales')
```

```
rollup_cols = get_column_names(table_name='rollup')
```

```
creators_cols = get_column_names(table_name='creators')
```

```
teams_df_cols = get_column_names(table_name='teams_df')
```

```
cases_cols = get_column_names(table_name='cases')
```

```
season_stats_df_cols = get_column_names(table_name='season_stats_df')
```

```
spotify_features_cols = get_column_names(table_name='spotify_features')
```

```
death_data_cols = get_column_names(table_name='death_data')
```

```
judges_cols = get_column_names(table_name='judges')
```

```
songs_cols = get_column_names(table_name='songs')
```

```
referees_df_cols = get_column_names(table_name='referees_df')
```

```
sales_cols = get_column_names(table_name='sales')
```

```
venue_cols = get_column_names(table_name='venue')
```

```
event_cols = get_column_names(table_name='event')
```

```
users_cols = get_column_names(table_name='users')
```

```
category_cols = get_column_names(table_name='category')
```

```
listing_cols = get_column_names(table_name='listing')
```

```
date_cols = get_column_names(table_name='date')
```

```
game_data_df_cols = get_column_names(table_name='game_data_df')
```

```
movies_cols = get_column_names(table_name='movies')
```

```
ratings_cols = get_column_names(table_name='ratings')
```

```
casetyp_cols = get_column_names(table_name='casetyp')
```

```
# ...
# ...
# ...
# ...
```

```
# Could also just use a list comprehension vs a bunch of print statements :)
```

```
names = get_table_names()
```

```
[get_column_names(table_name= x) for x in names]
```

```
[[('movieid',), ('domestic',), ('international',)],
 [('total_cases',),
  ('num_lib_decisions',),
  ('percent_liberal',),
  ('party_name',),
  ('category_name',)],
 [('lgbtq',),
  ('age',),
  ('famous',),
  ('creator_id',),
  ('gender',),
  ('genre',),
  ('ethnicity',),
  ('creator',),
  ('username',),
  ('country',)],
 [('Team ID',), ('Team Name',)],
 [('case_id',),
  ('judge_id',),
  ('case_year',),
  ('casetype_id',),
  ('category_id',),
  ('libcon_id',),
  ('libcon_name',),
  ('category_name',)],
 [('Non-Penalty Expected Goals + Assisted Goals per Game',),
  ('Team ID',),
  ('Rank',),
  ('Wins',),
  ('Draws',),
  ('Losses',),
  ('Goals For',),
  ('Goals Against',),
  ('Goal Difference',),
  ('Points',),
  ('Points Per Match',),
  ('Number of Players',),
  ('Age',),
  ('Possession',),
  ('Goals Scored',),
  ('Assists',),
  ('Goals + Assists',),
  ('Non-Penalty Expected Goals + Assisted Goals per Game',)]
```

```
( 'Non-Penalty Goals' ),),
('Penalties Made',),
('Penalties Attempted',),
('Penalty Conversion Rate',),
('Yellow Cards',),
('Red Cards',),
('Goals per Game',),
('Assists per Game',),
('Goals + Assists per Game',),
('Non-Penalty Goals per Game',),
('Non-Penalty Goals + Assists per Game',),
('Expected Goals',),
('Expected Goals Against',),
('Expected Goal Difference',),
('Expected Goal Difference per Game',),
('Non-Penalty Expected Goals',),
```

```
# Query on events
```

```
sq = """SELECT * FROM event LIMIT 5"""
```

```
run_query(sq)
```

```
(['event_id', 'venue_id', 'cat_id', 'date_id', 'event_name', 'start_time'],
 [(2, 306, 8, 2114, 'Boris Godunov', datetime.datetime(2008, 10, 15, 20, 0)),
 (3, 302, 8, 1935, 'Salome', datetime.datetime(2008, 4, 19, 14, 30)),
 (4,
 309,
 8,
 2090,
 'La Cenerentola (Cinderella)',
 datetime.datetime(2008, 9, 21, 14, 30)),
 (5, 302, 8, 1982, 'Il Trovatore', datetime.datetime(2008, 6, 5, 19, 0)),
 (6,
 308,
 8,
 2109,
 'L Elisir d Amore',
 datetime.datetime(2008, 10, 10, 19, 30))])
```

```
# Use sql_head to get the head of sales
```

```
sql_head(table_name='sales')
```

```
# Debugging
```

```
sql_head(table_name='users')
```

```
## Q2 Your function ends here - Any code outside of these start/end markers won't be grad
```

	user_id	user_name	first_name	last_name	city	state	
0	2	PGL08LJI	Vladimir	Humphrey	Murfreesboro	SK	Suspendisse.tristi
1	3	IFT66TXU	Lars	Ratliff	High Point	ME	amet.faucibus.ut@con

2	4	XDZ38RDD	Barry	Roy	Omaha	AB	
3	5	AEB55QTM	Reagan	Hodge	Forest Lake	NS	
4	6	NDQ15VBM	Victor	Hernandez	Naperville	GA	turp

Q3 WHERE - 5 points

Now let's do a bit of filtering with WHERE. Write and run queries to get the following results.

LIMIT all returns to first five rows.

- Get venues (**full row of data, including venue_id, etc.**) with ≥ 10000 seats from the venues table
- Get venues (**full row of data, including venue_id, etc.**) in Arizona
- Get users (**just the first names**) who have a first name that starts with H
- Get the email addresses (**just the email addresses**) of users who gave a .edu email address

Make sure to use the sq1, sq2, and sq3 as variables to store your queries in because those will specifically be evaluated

[] ↪ 4 cells hidden

Q4 GROUP BY and HAVING - 5 points

Time to practice some GROUP BY and HAVING operations. Please write and run queries that do the following:

GROUP BY application

- Find the top five venues that hosted the most events: Alias the count of events as 'events_hosted'. Also return the venue ID
- Get the number of events hosted in each month. You'll need to use `date_part()` in your select to select just the months. Alias this as 'month' and then the count of the number of events hosted as 'events_hosted'.
- Get the top five sellers who made the most commission. Alias their total commission made as 'total_com'. Also get their average commission made and alias as 'avg_com'. Be sure to also display the seller_id.

HAVING application

HAVING application

- Using the same query as the last one, instead of getting the top five sellers get all sellers who have made a total commission greater than \$4000.
- Using the same query as the first groupby, instead of returning the top five venues, return just the ID's of venues that have had greater than 60 events.

Make sure to use the sq1, sq2, and sq3 as variables to store your queries in because those will specifically be evaluated

[] ↳ 5 cells hidden

Q5 JOIN - 5 points

Time for some joins. You've probably noticed by now that there is at least one relational key in each table, but some have more. For example, sales has a unique sale id, listing id, seller id, buyer id, date id. This allows you to link each sale to relevant information in other tables.

LIMIT each output to 5

Please write queries to do the following items:

- Join information of users to each sale made.
- Join information about each venue to each event.

Make sure to use the sq1, sq2, sq3, and sq4 as variables to store your queries in because those will specifically be evaluated

[] ↳ 6 cells hidden

✓ Q 6 Subqueries - 5 points

To wrap up let's do several subqueries. Please do the following:

- Get all purchases made by users of live in Arizona
- Get event information for all events that took place in a venue where the venue name ends with 'Stadium'.
- Get event information for all events where the total ticket sales were greater than \$50,000.

Make sure to use the sq1, sq2, and sq3 as variables to store your queries in because those will specifically be evaluated

```
## Q6 Your function starts here
# Get all purchases from users who live in Arizona
```

```
# sql_head(table_name='users')
sql = """ SELECT * FROM sales
        WHERE buyer_id IN (SELECT user_id FROM users WHERE state = 'AZ')
        ;"""
run_query(sql)

(['sales_id',
 'list_id',
 'seller_id',
 'buyer_id',
 'event_id',
 'date_id',
 'qty_sold',
 'price_paid',
 'commission',
 'sale_time'],
[(43, 47, 49346, 33489, 8577, 2141, 2, 378, 56.7, '11/11/2008 09:51:06'),
 (79, 101, 37592, 7079, 3340, 1878, 1, 36, 5.4, '2/21/2008 10:32:10'),
 (81, 103, 26314, 7079, 15, 2033, 1, 181, 27.15, '7/26/2008 11:04:13'),
 (83, 106, 12538, 7079, 250, 1884, 1, 109, 16.35, '2/27/2008 11:58:35'),
 (154, 162, 27703, 46451, 2906, 1907, 3, 426, 63.9, '3/22/2008 05:21:40'),
 (173, 176, 3941, 10086, 3097, 2155, 2, 40, 6.0, '11/25/2008 09:20:09'),
 (218, 231, 21568, 41178, 1077, 2140, 2, 458, 68.7, '11/10/2008 04:12:53'),
 (260, 267, 23965, 7079, 4288, 2135, 2, 270, 40.5, '11/5/2008 11:27:05'),
 (327, 321, 44084, 41178, 1237, 1893, 2, 396, 59.4, '3/8/2008 10:10:23'),
 (368, 361, 36225, 7079, 7857, 2139, 2, 396, 59.4, '11/9/2008 03:12:00'),
 (591, 597, 4168, 3181, 8131, 2039, 4, 844, 126.6, '8/1/2008 05:38:55'),
 (717, 743, 6621, 24584, 2233, 1965, 4, 300, 45.0, '5/19/2008 06:42:34'),
 (767, 782, 24749, 21507, 2120, 1930, 2, 110, 16.5, '4/14/2008 11:32:51'),
 (860, 872, 15798, 40597, 2249, 1997, 4, 372, 55.8, '6/20/2008 08:57:51'),
 (989, 999, 8141, 3181, 7284, 2097, 4, 772, 115.8, '9/28/2008 07:48:18'),
 (1176, 1221, 233, 1285, 7973, 1860, 2, 320, 48.0, '2/3/2008 11:09:38'),
 (1306, 1380, 7634, 27494, 7982, 2057, 4, 564, 84.6, '8/19/2008 09:04:04'),
 (1572, 1678, 32145, 7079, 6646, 2086, 2, 414, 62.1, '9/17/2008 04:57:57'),
 (1951, 2097, 33908, 24449, 8536, 1909, 2, 434, 65.1, '3/24/2008 03:05:31'),
 (2053, 2198, 17797, 3181, 610, 2156, 4, 728, 109.2, '11/26/2008 08:31:54'),
 (2066, 2217, 42102, 1285, 8234, 1898, 2, 464, 69.6, '3/13/2008 09:40:34'),
 (2101, 2246, 48938, 39155, 1788, 2128, 1, 115, 17.25, '10/29/2008 12:05:45'),
 (2283, 2457, 46008, 26052, 1241, 2110, 3, 606, 90.9, '10/11/2008 09:31:50'),
 (2319, 2489, 31876, 1285, 2016, 1945, 2, 354, 53.1, '4/29/2008 11:26:23'),
 (2454, 2627, 38477, 1285, 4556, 1840, 2, 192, 28.8, '1/14/2008 06:54:01'),
 (2507, 2680, 37228, 33935, 2074, 1989, 3, 423, 63.45, '6/12/2008 10:06:55'),
 (2515, 2687, 29895, 27494, 722, 1881, 4, 776, 116.4, '2/24/2008 10:22:19'),
 (2550, 2736, 5999, 10086, 6778, 2149, 2, 60, 9.0, '11/19/2008 11:53:38'),
 (2611, 2798, 25075, 12900, 3514, 1989, 4, 508, 76.2, '6/12/2008 02:11:32'),
 (2722, 2930, 36164, 21507, 8124, 1953, 2, 174, 26.1, '5/7/2008 07:31:46'),
 (2808, 3003, 47329, 39155, 222, 1973, 1, 226, 33.9, '5/27/2008 12:11:31'),
 (2849, 3045, 9358, 21507, 1488, 1883, 2, 128, 19.2, '2/26/2008 01:58:10'),
 (2867, 3064, 37318, 10086, 2952, 1937, 2, 296, 44.4, '4/21/2008 03:08:59'),
 (2970, 3166, 7451, 2921, 1560, 2088, 2, 68, 10.2, '9/19/2008 07:48:21'),
 (3058, 3276, 1599, 43099, 6700, 2075, 2, 334, 50.1, '9/6/2008 12:04:37'),
 (3094, 3314, 36103, 20334, 2948, 1904, 1, 25, 3.75, '3/19/2008 02:12:27'),
 (3151, 3377, 18670, 20334, 3875, 1914, 1, 250, 37.5, '3/29/2008 04:43:27'),
 (3174, 3406, 38736, 24584, 161, 2007, 4, 944, 141.6, '6/30/2008 05:37:39'),
 (3238, 3484, 47920, 21784, 4098, 2072, 4, 204, 30.6, '9/3/2008 08:06:35'),
```

```
(3288, 3555, 27182, 22897, 1165, 1911, 1, 35, 5.25, '3/26/2008 10:19:25'),
(3514, 3785, 33751, 2508, 3882, 2050, 1, 75, 11.25, '8/12/2008 07:32:39'),
(3531, 3810, 36154, 2508, 3215, 2150, 1, 244, 36.6, '11/20/2008 08:27:30'),
(3682, 3982, 30455, 24584, 859, 2107, 4, 356, 53.4, '10/8/2008 02:34:06'),
(3708, 4009, 24486, 813, 1361, 2066, 2, 190, 28.5, '8/28/2008 03:48:36'),
(3864, 4166, 13860, 34913, 1019, 2179, 2, 220, 33.0, '12/19/2008 10:02:22'),
(3911, 4207, 10339, 2508, 156, 2026, 1, 143, 21.45, '7/19/2008 11:35:19'),
(4066, 4362, 24791, 21131, 4182, 2016, 4, 532, 79.8, '7/9/2008 05:26:09'),
(4122, 4431, 44033, 2921, 7355, 2115, 2, 224, 33.6, '10/16/2008 08:10:02'),
```

```
# Get event information for all events that took place in a venue where the name ended in
sq2 = """ SELECT * FROM event
        WHERE venue_id IN (SELECT venue_id FROM venue WHERE venue_name LIKE '%Stadium')
        ;"""
run_query(sq2)
```

```
(['event_id', 'venue_id', 'cat_id', 'date_id', 'event_name', 'start_time'],
[(3803,
  2,
  9,
  2181,
  'Dropkick Murphys',
  datetime.datetime(2008, 12, 21, 14, 0)),
(3816, 11, 9, 2139, 'Keb Mo', datetime.datetime(2008, 11, 9, 19, 0)),
(3821,
  79,
  9,
  1885,
  'Charlie Daniels Band',
  datetime.datetime(2008, 2, 28, 19, 30)),
(3824, 98, 9, 1885, 'Govt Mule', datetime.datetime(2008, 2, 28, 14, 0)),
(3835, 74, 9, 2073, 'LeAnn Rimes', datetime.datetime(2008, 9, 4, 14, 30)),
(3837, 67, 9, 1850, 'Randy Travis', datetime.datetime(2008, 1, 24, 15, 0)),
(3840, 14, 9, 1876, 'Gwen Stefani', datetime.datetime(2008, 2, 19, 19, 30)),
(3843, 75, 9, 1986, 'Blake Shelton', datetime.datetime(2008, 6, 9, 14, 30)),
(3854, 6, 9, 1982, 'Gwen Stefani', datetime.datetime(2008, 6, 5, 15, 0)),
(3862, 116, 9, 1855, 'Teddy Geiger', datetime.datetime(2008, 1, 29, 19, 30)),
(3864, 75, 9, 1992, 'Celine Dion', datetime.datetime(2008, 6, 15, 15, 0)),
(3866, 72, 9, 1950, 'Cold War Kids', datetime.datetime(2008, 5, 1, 14, 0)),
(3869, 67, 9, 2115, 'Dave Stewart', datetime.datetime(2008, 10, 16, 20, 0)),
(3873, 11, 9, 2188, 'Avril Lavigne', datetime.datetime(2008, 12, 28, 14, 0)),
(3874, 129, 9, 1885, 'America', datetime.datetime(2008, 2, 28, 15, 0)),
(3889,
  67,
  9,
  2108,
  'Natasha Bedingfield',
  datetime.datetime(2008, 10, 9, 19, 30)),
(3896,
  14,
  9,
  1893,
  'Natasha Bedingfield',
  datetime.datetime(2008, 3, 8, 15, 0)),
(3900, 75, 9, 1986, 'Blake Shelton', datetime.datetime(2008, 6, 9, 14, 30)),
```

```
(3899, 15, 9, 2089, 'Rusted Root', datetime.datetime(2008, 9, 20, 14, 0)),
(3901, 70, 9, 1861, 'Squeeze', datetime.datetime(2008, 2, 4, 14, 0)),
(3908,
 74,
 9,
 2098,
 'Sound Tribe Sector 9',
 datetime.datetime(2008, 9, 29, 14, 0)),
(3909, 72, 9, 2190, 'Scorpions', datetime.datetime(2008, 12, 30, 14, 0)),
(3912, 103, 9, 1926, 'Nofx', datetime.datetime(2008, 4, 10, 19, 0)),
(3918, 74, 9, 1940, 'Gnarls Barkley', datetime.datetime(2008, 4, 24, 15, 0)),
(3925, 3, 9, 1864, 'Donny Osmond', datetime.datetime(2008, 2, 7, 14, 30)),
(3926, 72, 9, 1848, 'Neil Diamond', datetime.datetime(2008, 1, 22, 19, 30)),
(3927, 76, 9, 1925, 'Marc Anthony', datetime.datetime(2008, 4, 9, 14, 30)),
(3935,
 116,
 9,
 2041,
 'Los Lonely Boys',
 datetime.datetime(2008, 8, 3, 15, 0)),
```

```
# Get event information where the total sales for that event were greater than $50000
# sql_head(table_name='sales')
sq3 = """ SELECT * FROM event
          WHERE event_id IN (SELECT event_id FROM sales GROUP BY event_id HAVING SUM(pric
;"""
```

```
run_query(sq3)
```

```
(['event_id', 'venue_id', 'cat_id', 'date_id', 'event_name', 'start_time'],
 [(289,
 300,
 8,
 2100,
 'Adriana Lecouvreur',
 datetime.datetime(2008, 10, 1, 19, 30)),
 (1602,
 257,
 6,
 2128,
 'Phantom of the Opera',
 datetime.datetime(2008, 10, 29, 19, 30)),
 (7895, 47, 9, 1944, 'Janet Jackson', datetime.datetime(2008, 4, 28, 15, 0))])
```

```
get_table_names()
```

```
(('movie_sales',),
 ('rollup',),
 ('creators',),
 ('teams_df',),
 ('cases',),
 ('season_stats_df',),
 ('spotify_features',),
 ('death_data',),
 ('indices',))
```

```
\ judges',),
('songs',),
('referees_df',),
('sales',),
('venue',),
('event',),
('users',),
('category',),
('listing',),
('date',),
('game_data_df',),
('movies',),
('ratings',),
('casetype',)]
```

```
get_column_names(table_name= 'date')
```

```
[('date_id',),
 ('qtr',),
 ('year',),
 ('holiday',),
 ('week',),
 ('cal_date',),
 ('day',),
 ('month',)]
```

```
## Q6 Your function ends here - Any code outside of these start/end markers won't be graded
```